

LIN7216 – Formal Linguistics

Arguments

Adèle Hénot-Mortier

01/10/2026

Queen Mary University of London

Brainstorming

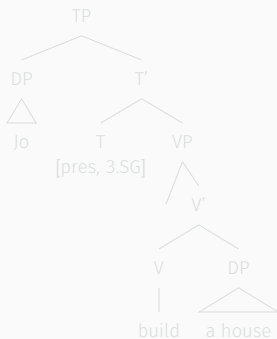
The arguments of a sentence

- How are subject, verb and object ordered? Is it fixed? Variable?
- How many subjects can a sentence have? How many objects?
- Is the subject of a sentence always “active”?

Building transitive sentences

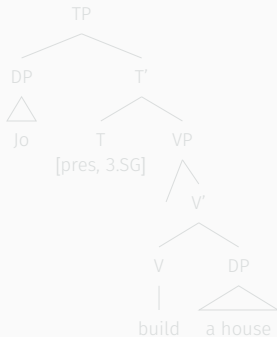
The SVO pattern

- We often think of English sentences as being **Subject-Verb-Object (SVO)**.
- Such sentences are built around **transitive** verbs: verbs that accept a complement (“Object”).
- Verbs that *require* a complement, like *devour*, *build*, *say*, are sometimes called **strongly transitive**.



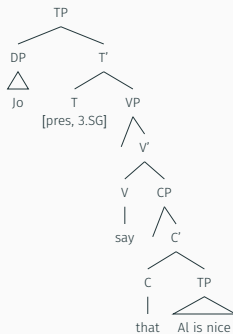
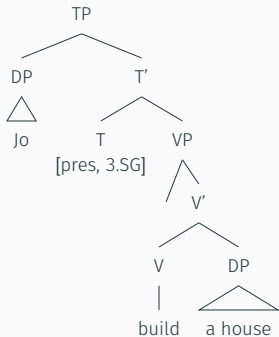
The SVO pattern

- We often think of English sentences as being **Subject-Verb-Object (SVO)**.
- Such sentences are built around **transitive** verbs: verbs that accept a complement (“Object”).
- Verbs that *require* a complement, like *devour*, *build*, *say*, are sometimes called **strongly transitive**.



The SVO pattern

- We often think of English sentences as being **Subject-Verb-Object (SVO)**.
- Such sentences are built around **transitive** verbs: verbs that accept a complement (“Object”).
- Verbs that *require* a complement, like *devour*, *build*, *say*, are sometimes called **strongly transitive**.



Word order is cross-linguistically variable

- Not all languages are SVO! Maybe you speak a language with a different canonical word order?

Word order is cross-linguistically variable

- Not all languages are SVO! Maybe you speak a language with a different canonical word order? This [map](#) shows the various word orders of the languages of the world.

Word order is cross-linguistically variable

- Not all languages are SVO! Maybe you speak a language with a different canonical word order? This [map](#) shows the various word orders of the languages of the world.
- But there are strong trends. How would you describe the first 2 lines of this table (76.5% of all languages)? The first 3 (83.4%)?

Word order	% languages	Examples
SOV	41	Turkish, Aleut, Basque (isolate)
SVO	35.5	Wolof (Niger-Congo), Breton, Guaraní (Tupian)
VSO	6.9	Irish, Tagalog (Austronesian), Standard Arabic
VOS	1.8	Lower Chinook, Malagasy (Austronesian), Kipea (isolate)
OVS	0.8	Tuvaluan (Austronesian), Tuyuca (Tucanoan)
OSV	0.3	Kxoe (Khoe-Kwadi), Wik Ngathana (Australian)
No dominant	13.7	Dinka (Nilo-Saharan), Cree (Algic)

Word order is cross-linguistically variable

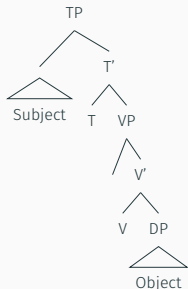
- Not all languages are SVO! Maybe you speak a language with a different canonical word order? This **map** shows the various word orders of the languages of the world.
- But there are strong trends. How would you describe the first 2 lines of this table (76.5% of all languages)? The first 3 (83.4%)?

Word order	% languages	Examples
SOV	41	Turkish, Aleut, Basque (isolate)
SVO	35.5	Wolof (Niger-Congo), Breton, Guaraní (Tupian)
VSO	6.9	Irish, Tagalog (Austronesian), Standard Arabic
VOS	1.8	Lower Chinook, Malagasy (Austronesian), Kipea (isolate)
OVS	0.8	Tuvaluan (Austronesian), Tuyuca (Tucanoan)
OSV	0.3	Kxoe (Khoe-Kwadi), Wik Ngathana (Australian)
No dominant	13.7	Dinka (Nilo-Saharan), Cree (Algic)

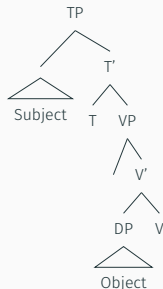
- It seems like most languages **start with the Subject**, or, at least, are s.t. **Subject precedes Object**.

Making sense of the 2 most common word orders

- Subject-first word orders make sense if we assume specifiers are pronounced (“**linearize**”) to the left of the phrase, while heads can linearize to the left or to the right of their complement.



SVO (V is **head-initial**)

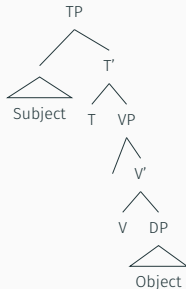


SOV (V is **head-final**)

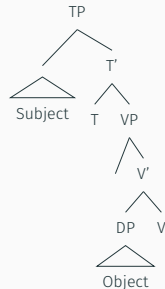
- How heads and their complement (in particular verbs and objects) are ordered is a syntactic parameter called **head-directionality**. It varies cross-linguistically.

Making sense of the 2 most common word orders

- Subject-first word orders make sense if we assume specifiers are pronounced (“**linearize**”) to the left of the phrase, while heads can linearize to the left or to the right of their complement.



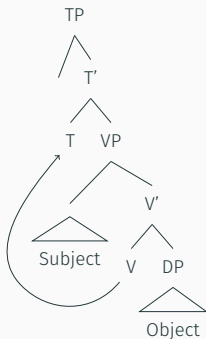
SVO (V is **head-initial**)



SOV (V is **head-final**)

- How heads and their complement (in particular verbs and objects) are ordered is a syntactic parameter called **head-directionality**. It varies cross-linguistically.

Extra: What about VSO?



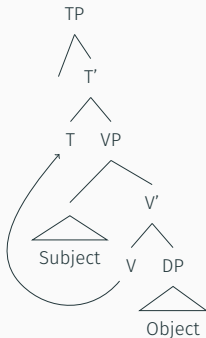
VSO

(VP-internal subject, V
head-moves to T)

- VSO is less straightforward than S-first orders because **the Subject seemingly intervenes between the V-head and its complement** (the Object)–contradicting the idea that heads and their complement are sisters.
- Independently motivated solution: the subject originates lower, in the **specifier of VP** (and stays there!), while the verb moves up to the T-head (in a process called **head movement**).
- The other (much less frequent) orders are trickier to explain.¹

¹VOS and OVS can be captured by moving the entire VP (V+O) or by assuming rightward specifiers. OSV may be captured assuming object movement.

Extra: What about VSO?



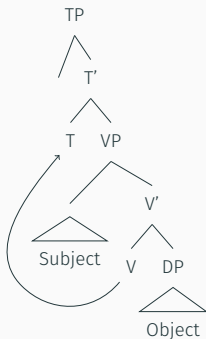
VSO

(VP-internal subject, V
head-moves to T)

- VSO is less straightforward than S-first orders because **the Subject seemingly intervenes between the V-head and its complement** (the Object)–contradicting the idea that heads and their complement are sisters.
- Independently motivated solution: the subject originates lower, in the **specifier of VP** (and stays there!), while the verb moves up to the T-head (in a process called **head movement**).
- The other (much less frequent) orders are trickier to explain.¹

¹VOS and OVS can be captured by moving the entire VP (V+O) or by assuming rightward specifiers. OSV may be captured assuming object movement.

Extra: What about VSO?



VSO

(VP-internal subject, V
head-moves to T)

- VSO is less straightforward than S-first orders because **the Subject seemingly intervenes between the V-head and its complement** (the Object)–contradicting the idea that heads and their complement are sisters.
- Independently motivated solution: the subject originates lower, in the **specifier of VP** (and stays there!), while the verb moves up to the T-head (in a process called **head movement**).
- The other (much less frequent) orders are trickier to explain.¹

¹VOS and OVS can be captured by moving the entire VP (V+O) or by assuming rightward specifiers. OSV may be captured assuming object movement.

Head directionality beyond OV vs. VO

- Head-directionality is subject to **cross-linguistic variation**. Maybe you speak a “**head-final**” language in which heads systematically come to the right of their complements! English is quite strongly **head-initial**, with heads linearizing on the left.
- Though many languages have the same head-directionality across heads (C, T, V, P), i.e. have them all final or all initial, some languages (e.g. German, Mandarin, Gbe) display **mixed head directionality**, with some heads (e.g. V) being initial, and some others (e.g. P, D), final.²

(1) Gbe (Niger-Congo)

Asíbá xò àvò àmàmú mātàn-mātàn dé
Asiba buy-PERF cloth green odd DEF
'Asiba bought a specific ugly green cloth.'

²Mixed directionality is however constrained by the Final-over-Final condition (FOFC): (roughly) a head-final phrase cannot immediately dominate a head-initial phrase.

Head directionality beyond OV vs. VO

- Head-directionality is subject to **cross-linguistic variation**. Maybe you speak a “**head-final**” language in which heads systematically come to the right of their complements! English is quite strongly **head-initial**, with heads linearizing on the left.
- Though many languages have the same head-directionality across heads (C, T, V, P), i.e. have them all final or all initial, some languages (e.g. German, Mandarin, Gbe) display **mixed head directionality**, with some heads (e.g. V) being initial, and some others (e.g. P, D), final.²

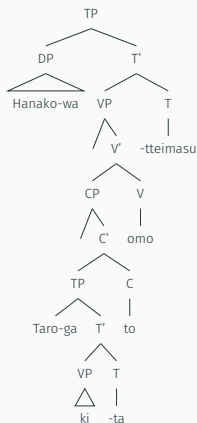
(1) Gbe (Niger-Congo)

Asíbá xò àvò àmàmú màtàn-màtàn dé
Asiba buy-PERF cloth green odd DEF
‘Asiba bought a specific ugly green cloth.’

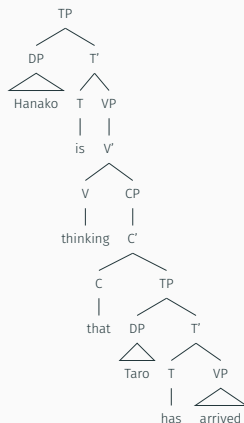
²Mixed directionality is however constrained by the Final-over-Final condition (**FOFC**): (roughly) a head-final phrase cannot immediately dominate a head-initial phrase.

English vs. Japanese

- (2) Hanako wa Taro ga ki-ta to omo-tteimasu.
Hanako TOP Taro NOM come-PST that think-IPFV.POL
(Politely) 'Hanako thinks that Taro has arrived.'



Strongly **head-final**



Strongly **head-initial**

Extra: the special case of V2

- Germanic languages³ are famously verb-second (“V2”), no matter which constituent comes first, in monoclausal (“**matrix**”) sentences.
- In **embedded** clauses however, all verbal material (auxiliaries included) come last, i.e. the word order is SOV.

(3) Gestern dachte Jo, dass er gute Fortschritte gemacht hatte.
Yesterday thought Jo, that he good progress done had.
‘Yesterday Jo thought he had made good progress.’

³but also Breton, Estonian, Kashmiri, Nawat, O’odham

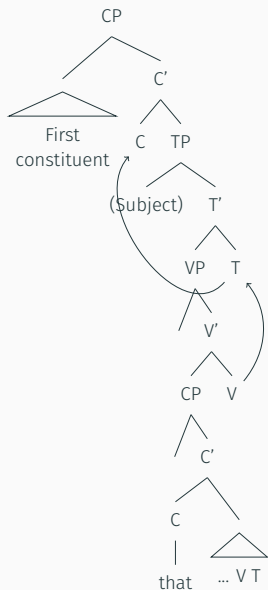
Extra: the special case of V2

- Germanic languages³ are famously verb-second (“V2”), no matter which constituent comes first, in monoclausal (“**matrix**”) sentences.
- In **embedded** clauses however, all verbal material (auxiliaries included) come last, i.e. the word order is SOV.

(3) Gestern dachte Jo, dass er gute Fortschritte gemacht hatte.
Yesterday thought Jo, that he good progress done had.
‘Yesterday Jo thought he had made good progress.’

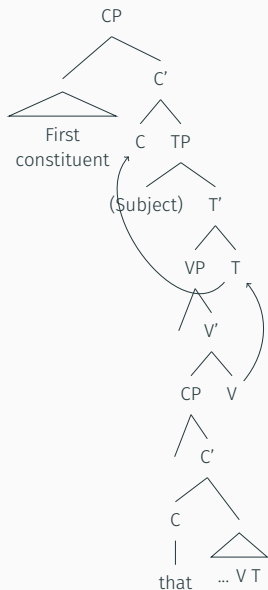
³but also Breton, Estonian, Kashmiri, Nawat, O’odham

Extra: the special case of V2 cont'd



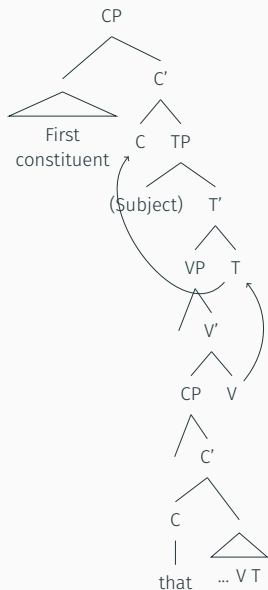
- A solution to the V2 puzzle is to assume SOV (i.e. head-final Ts and Vs), with matrix clauses undergoing **2 obligatory movement steps**:
 - Movement of **some constituent** (Subject or non-Subject) to the **specifier** of a high topic phrase/CP.
 - Movement of **the main verb/auxiliary** to the **head** of that topic phrase/CP.
- The presence of an overt C in embedded clauses blocks these, revealing the underlying word order.
- This makes V2 languages like German **mixed head-directionality** (T/V final, other heads initial).

Extra: the special case of V2 cont'd



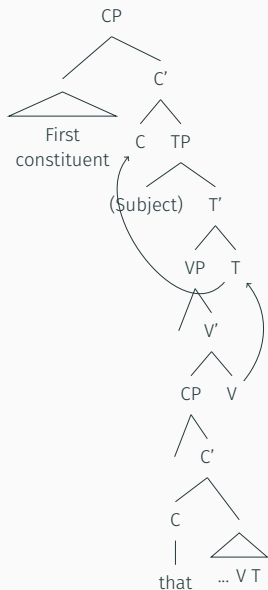
- A solution to the V2 puzzle is to assume SOV (i.e. head-final Ts and Vs), with matrix clauses undergoing **2 obligatory movement steps**:
 - Movement of **some constituent** (Subject or non-Subject) to the **specifier** of a high topic phrase/CP.
 - Movement of **the main verb/auxiliary** to the head of that topic phrase/CP.
- The presence of an overt C in embedded clauses blocks these, revealing the underlying word order.
- This makes V2 languages like German **mixed head-directionality** (T/V final, other heads initial).

Extra: the special case of V2 cont'd



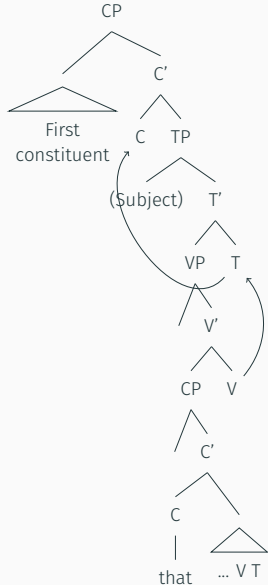
- A solution to the V2 puzzle is to assume SOV (i.e. head-final Ts and Vs), with matrix clauses undergoing **2 obligatory movement steps**:
 - Movement of **some constituent** (Subject or non-Subject) to the **specifier** of a high topic phrase/CP.
 - Movement of **the main verb/auxiliary** to the **head** of that topic phrase/CP.
- The presence of an overt C in embedded clauses blocks these, revealing the underlying word order.
- This makes V2 languages like German **mixed head-directionality** (T/V final, other heads initial).

Extra: the special case of V2 cont'd



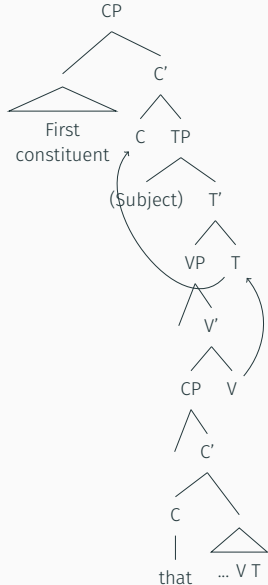
- A solution to the V2 puzzle is to assume SOV (i.e. head-final Ts and Vs), with matrix clauses undergoing **2 obligatory movement steps**:
 - Movement of **some constituent** (Subject or non-Subject) to the **specifier** of a high topic phrase/CP.
 - Movement of **the main verb/auxiliary** to the **head** of that topic phrase/CP.
- The presence of an overt C in embedded clauses blocks these, revealing the underlying word order.
- This makes V2 languages like German **mixed head-directionality** (T/V final, other heads initial).

Extra: the special case of V2 cont'd



- A solution to the V2 puzzle is to assume SOV (i.e. head-final Ts and Vs), with matrix clauses undergoing **2 obligatory movement steps**:
 - Movement of **some constituent** (Subject or non-Subject) to the **specifier** of a high topic phrase/CP.
 - Movement of **the main verb/auxiliary** to the **head** of that topic phrase/CP.
- The presence of an overt C in embedded clauses blocks these, revealing the underlying word order.
- This makes V2 languages like German **mixed head-directionality** (T/V final, other heads initial).

Extra: the special case of V2 cont'd



- A solution to the V2 puzzle is to assume SOV (i.e. head-final Ts and Vs), with matrix clauses undergoing **2 obligatory movement steps**:
 - Movement of **some constituent** (Subject or non-Subject) to the **specifier** of a high topic phrase/CP.
 - Movement of **the main verb/auxiliary** to the **head** of that topic phrase/CP.
- The presence of an overt C in embedded clauses blocks these, revealing the underlying word order.
- This makes V2 languages like German **mixed head-directionality** (T/V final, other heads initial).

Interperting transitive sentences

Simplest model: individuals and one-place predicates

- From last week: we can build a simple semantic formalism (**metalanguage**) based on **set theory**.
- We posit **individuals** and **sets of individuals** as primitive types of the metalanguage.
- A proper name like *Jo* (a syntactic object from the **object language** that is being interpreted) can be taken to denote the *individual* *Jo*, noted *j* (metalanguage). We write $\llbracket Jo \rrbracket = j$ (“the denotation of the word *Jo* is the individual *j*”).
- Similarly, an intransitive predicate like *smile* (object language) can be taken to denote the set of individuals who smile, noted $\{x \mid x \text{ smiles } \}$.

Simplest model: individuals and one-place predicates

- From last week: we can build a simple semantic formalism (**metalanguage**) based on **set theory**.
- We posit **individuals** and **sets of individuals** as primitive types of the metalanguage.
- A proper name like *Jo* (a syntactic object from the **object language** that is being interpreted) can be taken to denote the *individual* *Jo*, noted *j* (metalanguage). We write $\llbracket Jo \rrbracket = j$ ("the denotation of the word *Jo* is the individual *j*").
- Similarly, an intransitive predicate like *smile* (object language) can be taken to denote the set of individuals who smile, noted $\{x \mid x \text{ smiles } \}$.

Simplest model: individuals and one-place predicates

- From last week: we can build a simple semantic formalism (**metalanguage**) based on **set theory**.
- We posit **individuals** and **sets of individuals** as primitive types of the metalanguage.
- A proper name like *Jo* (a syntactic object from the **object language** that is being interpreted) can be taken to denote the *individual* *Jo*, noted *j* (metalanguage). We write $\llbracket Jo \rrbracket = j$ (“the denotation of the word *Jo* is the individual *j*”).
- Similarly, an intransitive predicate like *smile* (object language) can be taken to denote the set of individuals who smile, noted $\{x \mid x \text{ smiles}\}$.

Simplest model: individuals and one-place predicates

- From last week: we can build a simple semantic formalism (**metalanguage**) based on **set theory**.
- We posit **individuals** and **sets of individuals** as primitive types of the metalanguage.
- A proper name like *Jo* (a syntactic object from the **object language** that is being interpreted) can be taken to denote the *individual* *Jo*, noted *j* (metalanguage). We write $\llbracket Jo \rrbracket = j$ (“the denotation of the word *Jo* is the individual *j*”).
- Similarly, an intransitive predicate like *smile* (object language) can be taken to denote the set of individuals who smile, noted $\{x \mid x \text{ smiles } \}$.

Recap: the meaning of simple intransitive sentences

- A sentence like *Jo smiles* can then be derived by interpreting its two words, and assuming that they semantically combine via the set membership relation.

$$\begin{array}{c} \in \\ \swarrow \quad \searrow \\ \llbracket \text{Jo} \rrbracket \quad \llbracket \text{smile} \rrbracket \end{array} \quad \begin{array}{l} \llbracket \text{Jo smile} \rrbracket = 1 \text{ iff } \llbracket \text{Jo} \rrbracket \in \llbracket \text{smile} \rrbracket \\ = 1 \text{ iff } j \in \{x \mid x \text{ smiles} \} \\ = 1 \text{ iff } j \text{ smiles} \end{array}$$

- This is nice because **compositional**, but a bit hard to generalize to transitive sentences with two arguments, a subject and an object.⁴ Relatedly, the assumption that branching amounts to set membership is also a bit arbitrary!

⁴We could say that a predicate such as *like* denotes the set of *pairs* of individuals s.t. the 1st element likes the 2nd element. $[_{TP} \text{Jo} [_{VP} \text{likes Al}]]$ would then be true iff the pair (j, a) belongs to that set. But this analysis does not seem to follow the basic parse of the sentence, as it suggests *Jo* and *Al* merge together to be interpreted as a pair (via a newly posited semantic rule), before combining with *like* via the set membership rule.

Recap: the meaning of simple intransitive sentences

- A sentence like *Jo smiles* can then be derived by interpreting its two words, and assuming that they semantically combine via the set membership relation.

$$\begin{array}{c} \in \\ \swarrow \quad \searrow \\ \llbracket \text{Jo} \rrbracket \quad \llbracket \text{smile} \rrbracket \end{array} \quad \begin{array}{l} \llbracket \text{Jo smile} \rrbracket = 1 \text{ iff } \llbracket \text{Jo} \rrbracket \in \llbracket \text{smile} \rrbracket \\ = 1 \text{ iff } j \in \{x \mid x \text{ smiles} \} \\ = 1 \text{ iff } j \text{ smiles} \end{array}$$

- This is nice because **compositional**, but a bit hard to generalize to transitive sentences with two arguments, a subject and an object.⁴ Relatedly, the assumption that branching amounts to set membership is also a bit arbitrary!

⁴We could say that a predicate such as *like* denotes the set of *pairs* of individuals s.t. the 1st element likes the 2nd element. $\llbracket_{TP} \text{Jo} \llbracket_{VP} \text{likes Al} \rrbracket \rrbracket$ would then be true iff the pair (j, a) belongs to that set. But this analysis does not seem to follow the basic parse of the sentence, as it suggests *Jo* and *Al* merge together to be interpreted as a pair (via a newly posited semantic rule), before combining with *like* via the set membership rule.

Transitioning to functions

- Let's briefly go back to intransitive, one-place predicates.
- In a set theoretic-framework, saying a predicate like *smile* applies to an individual like *j*, amounts to saying *j* belongs to set of smiling individuals.
- Let's introduce a useful bridge between **sets**, and (more expressive) **functions**. For any set *S*, we can define a **characteristic function** χ_S that applies to inputs whose type matches that of the members of *S* (think for now: individuals) and returns 1 iff the input belongs to *S*.

General definition:

$$\chi_S : X \mapsto \begin{cases} 1 & \text{if } x \in S \\ 0 & \text{otherwise} \end{cases}$$

Applied to $\llbracket \textit{smile} \rrbracket$:

$$\chi_{\llbracket \textit{smile} \rrbracket} : X \mapsto \begin{cases} 1 & \text{if } x \text{ smiles} \\ 0 & \text{otherwise} \end{cases}$$

Transitioning to functions

- Let's briefly go back to intransitive, one-place predicates.
- In a set theoretic-framework, saying a predicate like *smile* applies to an individual like *j*, amounts to saying *j* belongs to set of smiling individuals.
- Let's introduce a useful bridge between **sets**, and (more expressive) **functions**. For any set *S*, we can define a **characteristic function** χ_S that applies to inputs whose type matches that of the members of *S* (think for now: individuals) and returns 1 iff the input belongs to *S*.

General definition:

$$\chi_S : X \mapsto \begin{cases} 1 & \text{if } x \in S \\ 0 & \text{otherwise} \end{cases}$$

Applied to $\llbracket \text{smile} \rrbracket$:

$$\chi_{\llbracket \text{smile} \rrbracket} : X \mapsto \begin{cases} 1 & \text{if } x \text{ smiles} \\ 0 & \text{otherwise} \end{cases}$$

Transitioning to functions

- Let's briefly go back to intransitive, one-place predicates.
- In a set theoretic-framework, saying a predicate like *smile* applies to an individual like *j*, amounts to saying *j* belongs to set of smiling individuals.
- Let's introduce a useful bridge between **sets**, and (more expressive) **functions**. For any set *S*, we can define a **characteristic function** χ_S that applies to inputs whose type matches that of the members of *S* (think for now: individuals) and returns 1 iff the input belongs to *S*.

General definition:

$$\chi_S : X \mapsto \begin{cases} 1 & \text{if } x \in S \\ 0 & \text{otherwise} \end{cases}$$

Applied to $\llbracket \text{smile} \rrbracket$:

$$\chi_{\llbracket \text{smile} \rrbracket} : X \mapsto \begin{cases} 1 & \text{if } x \text{ smiles} \\ 0 & \text{otherwise} \end{cases}$$

Transitioning to functions

- Let's briefly go back to intransitive, one-place predicates.
- In a set theoretic-framework, saying a predicate like *smile* applies to an individual like *j*, amounts to saying *j* belongs to set of smiling individuals.
- Let's introduce a useful bridge between **sets**, and (more expressive) **functions**. For any set *S*, we can define a **characteristic function** χ_S that applies to inputs whose type matches that of the members of *S* (think for now: individuals) and returns 1 iff the input belongs to *S*.

General definition:

$$\chi_S : X \mapsto \begin{cases} 1 & \text{if } x \in S \\ 0 & \text{otherwise} \end{cases}$$

Applied to $\llbracket \text{smile} \rrbracket$:

$$\chi_{\llbracket \text{smile} \rrbracket} : X \mapsto \begin{cases} 1 & \text{if } x \text{ smiles} \\ 0 & \text{otherwise} \end{cases}$$

Transitioning to functions

- Let's briefly go back to intransitive, one-place predicates.
- In a set theoretic-framework, saying a predicate like *smile* applies to an individual like *j*, amounts to saying *j* belongs to set of smiling individuals.
- Let's introduce a useful bridge between **sets**, and (more expressive) **functions**. For any set *S*, we can define a **characteristic function** χ_S that applies to inputs whose type matches that of the members of *S* (think for now: individuals) and returns 1 iff the input belongs to *S*.

General definition:

$$\chi_S : X \mapsto \begin{cases} 1 & \text{if } x \in S \\ 0 & \text{otherwise} \end{cases}$$

Applied to $\llbracket \text{smile} \rrbracket$:

$$\chi_{\llbracket \text{smile} \rrbracket} : X \mapsto \begin{cases} 1 & \text{if } x \text{ smiles} \\ 0 & \text{otherwise} \end{cases}$$

The λ -notation

General definition:

$$\chi_S : x \mapsto \begin{cases} 1 & \text{if } x \in S \\ 0 & \text{otherwise} \end{cases}$$

Applied to $\llbracket \text{smile} \rrbracket$:

$$\chi_{\llbracket \text{smile} \rrbracket} : x \mapsto \begin{cases} 1 & \text{if } x \text{ smiles} \\ 0 & \text{otherwise} \end{cases}$$

- We have $x \in S \Leftrightarrow \chi_S(x) = 1$
- We will write functions using the lambda notation:
 $\chi_S = \lambda x. (1 \text{ if } x \in S \text{ and } 0 \text{ otherwise})$, and, for short,
 $\lambda x. (1 \text{ iff } x \in S)$.
- In that notation, the λ says “I’m a function”, whatever is between the λ and the $.$ is the function’s **input** (a *bound variable*), and whatever follows the $.$ is the function’s **body/output**.
- We now take the denotation of predicates to be characteristic functions in the λ -notation instead of sets: $\llbracket \text{smile} \rrbracket = \chi_{\text{smile}} = \lambda x. (1 \text{ iff } x \text{ smiles})$, which means, “I’m a function taking a variable x and returning 1 iff x smiles”.

The λ -notation

General definition:

$$\chi_S : X \mapsto \begin{cases} 1 & \text{if } x \in S \\ 0 & \text{otherwise} \end{cases}$$

Applied to $\llbracket \text{smile} \rrbracket$:

$$\chi_{\llbracket \text{smile} \rrbracket} : X \mapsto \begin{cases} 1 & \text{if } x \text{ smiles} \\ 0 & \text{otherwise} \end{cases}$$

- We have $x \in S \Leftrightarrow \chi_S(x) = 1$
- We will write functions using the lambda notation:
 $\chi_S = \lambda x. (1 \text{ if } x \in S \text{ and } 0 \text{ otherwise})$, and, for short,
 $\lambda x. (1 \text{ iff } x \in S)$.
- In that notation, the λ says “I’m a function”, whatever is between the λ and the $.$ is the function’s **input** (a *bound variable*), and whatever follows the $.$ is the function’s **body/output**.
- We now take the denotation of predicates to be characteristic functions in the λ -notation instead of sets: $\llbracket \text{smile} \rrbracket = \chi_{\text{smile}} = \lambda x. (1 \text{ iff } x \text{ smiles})$, which means, “I’m a function taking a variable x and returning 1 iff x smiles”.

The λ -notation

General definition:

$$\chi_S : x \mapsto \begin{cases} 1 & \text{if } x \in S \\ 0 & \text{otherwise} \end{cases}$$

Applied to $\llbracket \text{smile} \rrbracket$:

$$\chi_{\llbracket \text{smile} \rrbracket} : x \mapsto \begin{cases} 1 & \text{if } x \text{ smiles} \\ 0 & \text{otherwise} \end{cases}$$

- We have $x \in S \Leftrightarrow \chi_S(x) = 1$
- We will write functions using the lambda notation:
 $\chi_S = \lambda x. (1 \text{ if } x \in S \text{ and } 0 \text{ otherwise})$, and, for short,
 $\lambda x. (1 \text{ iff } x \in S)$.
- In that notation, the λ says “I’m a function”, whatever is between the λ and the $.$ is the function’s **input** (a *bound variable*), and whatever follows the $.$ is the function’s **body/output**.
- We now take the denotation of predicates to be characteristic functions in the λ -notation instead of sets: $\llbracket \text{smile} \rrbracket = \chi_{\text{smile}} = \lambda x. (1 \text{ iff } x \text{ smiles})$, which means, “I’m a function taking a variable x and returning 1 iff x smiles”.

The λ -notation

General definition:

$$\chi_S : x \mapsto \begin{cases} 1 & \text{if } x \in S \\ 0 & \text{otherwise} \end{cases}$$

Applied to $\llbracket \text{smile} \rrbracket$:

$$\chi_{\llbracket \text{smile} \rrbracket} : x \mapsto \begin{cases} 1 & \text{if } x \text{ smiles} \\ 0 & \text{otherwise} \end{cases}$$

- We have $x \in S \Leftrightarrow \chi_S(x) = 1$
- We will write functions using the lambda notation:
 $\chi_S = \lambda x. (1 \text{ if } x \in S \text{ and } 0 \text{ otherwise})$, and, for short,
 $\lambda x. (1 \text{ iff } x \in S)$.
- In that notation, the λ says “I’m a function”, whatever is between the λ and the $.$ is the function’s **input** (a *bound variable*), and whatever follows the $.$ is the function’s **body/output**.
- We now take the denotation of predicates to be characteristic functions in the λ -notation instead of sets: $\llbracket \text{smile} \rrbracket = \chi_{\text{smile}} = \lambda x. (1 \text{ iff } x \text{ smiles})$, which means, “I’m a function taking a variable x and returning 1 iff x smiles”.

Consequences for compositional interpretation

- Now that we are dealing with individuals and functions from individuals to truth values (instead of sets of individuals), we can have a more general semantics for how arguments and predicates combine: **Functional Application (FA)**.
- Functional Application consists in applying a function to a suitable input (of the right type).
- Formally, this amounts to replacing all occurrences of the bound variable in the body of the function (the x s below), with the actual input it is fed with (j below), and getting rid of the λ .



$$\begin{aligned} \llbracket \text{Jo smile} \rrbracket &= \llbracket \text{smile} \rrbracket(\llbracket \text{Jo} \rrbracket) \\ &= (\lambda x. 1 \text{ iff } x \text{ smiles})(j) \\ &= 1 \text{ iff } j \text{ smiles} \end{aligned}$$

Consequences for compositional interpretation

- Now that we are dealing with individuals and functions from individuals to truth values (instead of sets of individuals), we can have a more general semantics for how arguments and predicates combine: **Functional Application (FA)**.
- Functional Application consists in applying a function to a suitable input (of the right type).
- Formally, this amounts to replacing all occurrences of the bound variable in the body of the function (the x s below), with the actual input it is fed with (j below), and getting rid of the λ .



$$\begin{aligned} \llbracket \text{Jo smile} \rrbracket &= \llbracket \text{smile} \rrbracket(\llbracket \text{Jo} \rrbracket) \\ &= (\lambda x. 1 \text{ iff } x \text{ smiles})(j) \\ &= 1 \text{ iff } j \text{ smiles} \end{aligned}$$

Consequences for compositional interpretation

- Now that we are dealing with individuals and functions from individuals to truth values (instead of sets of individuals), we can have a more general semantics for how arguments and predicates combine: **Functional Application (FA)**.
- Functional Application consists in applying a function to a suitable input (of the right type).
- Formally, this amounts to replacing all occurrences of the bound variable in the body of the function (the x s below), with the actual input it is fed with (j below), and getting rid of the λ .



$$\begin{aligned} \llbracket \text{Jo smile} \rrbracket &= \llbracket \text{smile} \rrbracket (\llbracket \text{Jo} \rrbracket) \\ &= (\lambda x. 1 \text{ iff } x \text{ smiles})(j) \\ &= 1 \text{ iff } j \text{ smiles} \end{aligned}$$

Functions with more than 1 argument

- The big plus of functions is that they can easily take **more than one argument**. In the λ -notation, this is often done **sequentially**.

$\llbracket \text{smile} \rrbracket = \lambda x. (1 \text{ iff } x \text{ smiles})$

$\llbracket \text{like} \rrbracket = \lambda y. \lambda x. (1 \text{ iff } x \text{ likes } y)$

- The denotation of *like* reads literally as: a function that takes a variable x , and returns a function that takes a variable y , which returns 1 iff x likes y .
- Alternatively: a function that takes an x and then a y , and returns 1 iff x likes y .
- x and y are pure **placeholders**, all their occurrences can be swapped. However, the **order** in which they are fed to the function matters as it needs to match (syntactic) argument structure.

$\llbracket \text{like} \rrbracket = \lambda x. \lambda y. (1 \text{ iff } y \text{ likes } x)$
 $\neq \lambda y. \lambda x. (1 \text{ iff } y \text{ likes } x)$

Takes likee 1st ✓

Takes liker 1st ✗

Functions with more than 1 argument

- The big plus of functions is that they can easily take **more than one argument**. In the λ -notation, this is often done **sequentially**.

$\llbracket \text{smile} \rrbracket = \lambda x. (1 \text{ iff } x \text{ smiles})$

$\llbracket \text{like} \rrbracket = \lambda y. \lambda x. (1 \text{ iff } x \text{ likes } y)$

- The denotation of *like* reads literally as: a function that takes a variable x , and returns a function that takes a variable y , which returns 1 iff x likes y .
- Alternatively: a function that takes an x and then a y , and returns 1 iff x likes y .
- x and y are pure **placeholders**, all their occurrences can be swapped. However, the **order** in which they are fed to the function matters as it needs to match (syntactic) argument structure.

$\llbracket \text{like} \rrbracket = \lambda x. \lambda y. (1 \text{ iff } y \text{ likes } x)$
 $\neq \lambda y. \lambda x. (1 \text{ iff } y \text{ likes } x)$

Takes likee 1st ✓

Takes liker 1st ✗

Functions with more than 1 argument

- The big plus of functions is that they can easily take **more than one argument**. In the λ -notation, this is often done **sequentially**.

$\llbracket \text{smile} \rrbracket = \lambda x. (1 \text{ iff } x \text{ smiles})$

$\llbracket \text{like} \rrbracket = \lambda y. \lambda x. (1 \text{ iff } x \text{ likes } y)$

- The denotation of *like* reads literally as: a function that takes a variable x , and returns a function that takes a variable y , which returns 1 iff x likes y .
- Alternatively: a function that takes an x and then a y , and returns 1 iff x likes y .
- x and y are pure **placeholders**, all their occurrences can be swapped. However, the **order** in which they are fed to the function matters as it needs to match (syntactic) argument structure.

$\llbracket \text{like} \rrbracket = \lambda x. \lambda y. (1 \text{ iff } y \text{ likes } x)$
 $\neq \lambda y. \lambda x. (1 \text{ iff } y \text{ likes } x)$

Takes likee 1st ✓

Takes liker 1st ✗

Functions with more than 1 argument

- The big plus of functions is that they can easily take **more than one argument**. In the λ -notation, this is often done **sequentially**.

$\llbracket \text{smile} \rrbracket = \lambda x. (1 \text{ iff } x \text{ smiles})$

$\llbracket \text{like} \rrbracket = \lambda y. \lambda x. (1 \text{ iff } x \text{ likes } y)$

- The denotation of *like* reads literally as: a function that takes a variable x , and returns a function that takes a variable y , which returns 1 iff x likes y .
- Alternatively: a function that takes an x and then a y , and returns 1 iff x likes y .
- x and y are pure **placeholders**, all their occurrences can be swapped. However, the **order** in which they are fed to the function matters as it needs to match (syntactic) argument structure.

$\llbracket \text{like} \rrbracket = \lambda x. \lambda y. (1 \text{ iff } y \text{ likes } x)$
 $\neq \lambda y. \lambda x. (1 \text{ iff } y \text{ likes } x)$

Takes likee 1st ✓

Takes liker 1st ✗

Functions with more than 1 argument

- The big plus of functions is that they can easily take **more than one argument**. In the λ -notation, this is often done **sequentially**.

$\llbracket \text{smile} \rrbracket = \lambda x. (1 \text{ iff } x \text{ smiles})$

$\llbracket \text{like} \rrbracket = \lambda y. \lambda x. (1 \text{ iff } x \text{ likes } y)$

- The denotation of *like* reads literally as: a function that takes a variable x , and returns a function that takes a variable y , which returns 1 iff x likes y .
- Alternatively: a function that takes an x and then a y , and returns 1 iff x likes y .
- x and y are pure **placeholders**, all their occurrences can be swapped. However, the **order** in which they are fed to the function matters as it needs to match (syntactic) argument structure.

$\llbracket \text{like} \rrbracket = \lambda x. \lambda y. (1 \text{ iff } y \text{ likes } x)$
 $\neq \lambda y. \lambda x. (1 \text{ iff } y \text{ likes } x)$

Takes likee 1st ✓

Takes liker 1st ✗

Functions with more than 1 argument

- The big plus of functions is that they can easily take **more than one argument**. In the λ -notation, this is often done **sequentially**.

$\llbracket \text{smile} \rrbracket = \lambda x. (1 \text{ iff } x \text{ smiles})$

$\llbracket \text{like} \rrbracket = \lambda y. \lambda x. (1 \text{ iff } x \text{ likes } y)$

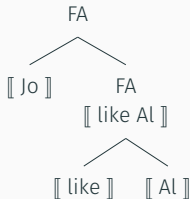
- The denotation of *like* reads literally as: a function that takes a variable x , and returns a function that takes a variable y , which returns 1 iff x likes y .
- Alternatively: a function that takes an x and then a y , and returns 1 iff x likes y .
- x and y are pure **placeholders**, all their occurrences can be swapped. However, the **order** in which they are fed to the function matters as it needs to match (syntactic) argument structure.

$\llbracket \text{like} \rrbracket = \lambda x. \lambda y. (1 \text{ iff } y \text{ likes } x)$
 $\neq \lambda y. \lambda x. (1 \text{ iff } y \text{ likes } x)$

Takes likee 1st ✓

Takes liker 1st ✗

Computing the meaning of a simple transitive sentence

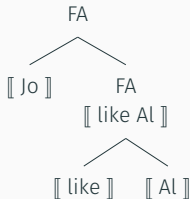


$$\begin{aligned} \llbracket \text{Jo like Al} \rrbracket &= (\llbracket \text{like} \rrbracket(\llbracket \text{Al} \rrbracket))(\llbracket \text{Jo} \rrbracket) \\ &= (\lambda y. \lambda x. 1 \text{ iff } x \text{ likes } y)(a)(j) \\ &= (\lambda x. 1 \text{ iff } x \text{ likes } a)(j) \\ &= 1 \text{ iff } j \text{ likes } a \end{aligned}$$

- Note Functional Application is **type-driven**:
 - Sometimes, the left element in the tree applies to the right element, as with $\llbracket \text{like} \rrbracket(\llbracket \text{Al} \rrbracket)$, because the left one is a function ranging over individuals⁵ and the right one is an individual.
 - Sometimes, as with $\llbracket \text{like Al} \rrbracket(\llbracket \text{Jo} \rrbracket)$, it's the other way around, because $\llbracket \text{like Al} \rrbracket$ is a function from individuals to truth values, while $\llbracket \text{like Al} \rrbracket$ is an individual.
- Also note that $\llbracket \text{like Al} \rrbracket = (\lambda x. 1 \text{ iff } x \text{ likes } a) \simeq$ the set of people who like Al is a one-place predicate: applying it to j brings us back to the intransitive case (like *Jo smiles*).

⁵And in this case returning functions from individual to truth values

Computing the meaning of a simple transitive sentence

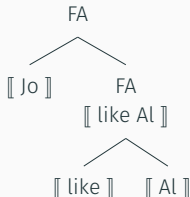


$$\begin{aligned} \llbracket \text{Jo like Al} \rrbracket &= (\llbracket \text{like} \rrbracket(\llbracket \text{Al} \rrbracket))(\llbracket \text{Jo} \rrbracket) \\ &= (\lambda y. \lambda x. 1 \text{ iff } x \text{ likes } y)(a)(j) \\ &= (\lambda x. 1 \text{ iff } x \text{ likes } a)(j) \\ &= 1 \text{ iff } j \text{ likes } a \end{aligned}$$

- Note Functional Application is **type-driven**:
 - Sometimes, the left element in the tree applies to the right element, as with $\llbracket \text{like} \rrbracket(\llbracket \text{Al} \rrbracket)$, because the left one is a function ranging over individuals⁵ and the right one is an individual.
 - Sometimes, as with $\llbracket \text{like Al} \rrbracket(\llbracket \text{Jo} \rrbracket)$, it's the other way around, because $\llbracket \text{like Al} \rrbracket$ is a function from individuals to truth values, while $\llbracket \text{like Al} \rrbracket$ is an individual.
- Also note that $\llbracket \text{like Al} \rrbracket = (\lambda x. 1 \text{ iff } x \text{ likes } a) \simeq$ the set of people who like Al is a one-place predicate: applying it to j brings us back to the intransitive case (like *Jo smiles*).

⁵And in this case returning functions from individual to truth values

Computing the meaning of a simple transitive sentence

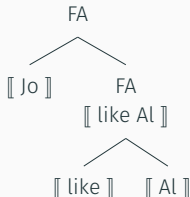


$$\begin{aligned} \llbracket \text{Jo like Al} \rrbracket &= (\llbracket \text{like} \rrbracket(\llbracket \text{Al} \rrbracket))(\llbracket \text{Jo} \rrbracket) \\ &= (\lambda y. \lambda x. 1 \text{ iff } x \text{ likes } y)(a)(j) \\ &= (\lambda x. 1 \text{ iff } x \text{ likes } a)(j) \\ &= 1 \text{ iff } j \text{ likes } a \end{aligned}$$

- Note Functional Application is **type-driven**:
 - Sometimes, the left element in the tree applies to the right element, as with $\llbracket \text{like} \rrbracket(\llbracket \text{Al} \rrbracket)$, because the left one is a function ranging over individuals⁵ and the right one is an individual.
 - Sometimes, as with $\llbracket \text{like Al} \rrbracket(\llbracket \text{Jo} \rrbracket)$, it's the other way around, because $\llbracket \text{like Al} \rrbracket$ is a function from individuals to truth values, while $\llbracket \text{like Al} \rrbracket$ is an individual.
- Also note that $\llbracket \text{like Al} \rrbracket = (\lambda x. 1 \text{ iff } x \text{ likes } a) \simeq$ the set of people who like Al is a one-place predicate: applying it to j brings us back to the intransitive case (like *Jo smiles*).

⁵And in this case returning functions from individual to truth values

Computing the meaning of a simple transitive sentence

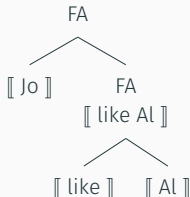


$$\begin{aligned} \llbracket \text{Jo like Al} \rrbracket &= (\llbracket \text{like} \rrbracket (\llbracket \text{Al} \rrbracket)) (\llbracket \text{Jo} \rrbracket) \\ &= (\lambda y. \lambda x. 1 \text{ iff } x \text{ likes } y)(a)(j) \\ &= (\lambda x. 1 \text{ iff } x \text{ likes } a)(j) \\ &= 1 \text{ iff } j \text{ likes } a \end{aligned}$$

- Note Functional Application is **type-driven**:
 - Sometimes, the left element in the tree applies to the right element, as with $\llbracket \text{like} \rrbracket (\llbracket \text{Al} \rrbracket)$, because the left one is a function ranging over individuals⁵ and the right one is an individual.
 - Sometimes, as with $\llbracket \text{like Al} \rrbracket (\llbracket \text{Jo} \rrbracket)$, it's the other way around, because $\llbracket \text{like Al} \rrbracket$ is a function from individuals to truth values, while $\llbracket \text{like Al} \rrbracket$ is an individual.
- Also note that $\llbracket \text{like Al} \rrbracket = (\lambda x. 1 \text{ iff } x \text{ likes } a) \simeq$ the set of people who like Al is a one-place predicate: applying it to j brings us back to the intransitive case (like *Jo smiles*).

⁵And in this case returning functions from individual to truth values

Computing the meaning of a simple transitive sentence



$$\begin{aligned} \llbracket \text{Jo like Al} \rrbracket &= (\llbracket \text{like} \rrbracket(\llbracket \text{Al} \rrbracket))(\llbracket \text{Jo} \rrbracket) \\ &= (\lambda y. \lambda x. 1 \text{ iff } x \text{ likes } y)(a)(j) \\ &= (\lambda x. 1 \text{ iff } x \text{ likes } a)(j) \\ &= 1 \text{ iff } j \text{ likes } a \end{aligned}$$

- Note Functional Application is **type-driven**:
 - Sometimes, the left element in the tree applies to the right element, as with $\llbracket \text{like} \rrbracket(\llbracket \text{Al} \rrbracket)$, because the left one is a function ranging over individuals⁵ and the right one is an individual.
 - Sometimes, as with $\llbracket \text{like Al} \rrbracket(\llbracket \text{Jo} \rrbracket)$, it's the other way around, because $\llbracket \text{like Al} \rrbracket$ is a function from individuals to truth values, while $\llbracket \text{like Al} \rrbracket$ is an individual.
- Also note that $\llbracket \text{like Al} \rrbracket = (\lambda x. 1 \text{ iff } x \text{ likes } a) \simeq$ the set of people who like Al is a one-place predicate: applying it to j brings us back to the intransitive case (like *Jo smiles*).

⁵And in this case returning functions from individual to truth values

Extra: more examples of functional meaning

- Suppose negation (expressed by *not* or the prefix *un-*) is a function taking a 1-place predicate and returning its negation $\llbracket \text{not} \rrbracket = \llbracket \text{un-} \rrbracket = \lambda P. \lambda x. 0 \text{ iff } P(x) = 1$ (“the function mapping a 1-place predicate P to the 1-place predicate reversing its truth-conditions”).

Function Application



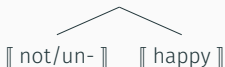
$$\begin{aligned}\llbracket \text{not happy} \rrbracket &= \llbracket \text{un-happy} \rrbracket = \llbracket \text{not} \rrbracket(\llbracket \text{happy} \rrbracket) \\ &= (\lambda P. \lambda x. 0 \text{ iff } P(x) = 1)(\lambda y. 1 \text{ iff } y \text{ is happy}) \\ &= \lambda x. 0 \text{ iff } (\lambda y. 1 \text{ iff } y \text{ is happy})(x) = 1 \\ &= \lambda x. 0 \text{ iff } ((1 \text{ iff } x \text{ is happy}) = 1) \\ &= \lambda x. 0 \text{ iff } x \text{ is happy}\end{aligned}$$

- This shows semantics can sometimes apply similarly **above and below the word**.

Extra: more examples of functional meaning

- Suppose negation (expressed by *not* or the prefix *un-*) is a function taking a 1-place predicate and returning its negation $\llbracket \text{not} \rrbracket = \llbracket \text{un-} \rrbracket = \lambda P. \lambda x. 0 \text{ iff } P(x) = 1$ (“the function mapping a 1-place predicate P to the 1-place predicate reversing its truth-conditions”).

Function Application



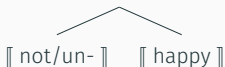
$$\begin{aligned}\llbracket \text{not happy} \rrbracket &= \llbracket \text{un-happy} \rrbracket = \llbracket \text{not} \rrbracket(\llbracket \text{happy} \rrbracket) \\ &= (\lambda P. \lambda x. 0 \text{ iff } P(x) = 1)(\lambda y. 1 \text{ iff } y \text{ is happy}) \\ &= \lambda x. 0 \text{ iff } (\lambda y. 1 \text{ iff } y \text{ is happy})(x) = 1 \\ &= \lambda x. 0 \text{ iff } ((1 \text{ iff } x \text{ is happy}) = 1) \\ &= \lambda x. 0 \text{ iff } x \text{ is happy}\end{aligned}$$

- This shows semantics can sometimes apply similarly **above and below the word**.

Extra: more examples of functional meaning

- Suppose negation (expressed by *not* or the prefix *un-*) is a function taking a 1-place predicate and returning its negation $\llbracket \text{not} \rrbracket = \llbracket \text{un-} \rrbracket = \lambda P. \lambda x. 0 \text{ iff } P(x) = 1$ (“the function mapping a 1-place predicate P to the 1-place predicate reversing its truth-conditions”).

Function Application



$$\begin{aligned}\llbracket \text{not happy} \rrbracket &= \llbracket \text{un-happy} \rrbracket = \llbracket \text{not} \rrbracket(\llbracket \text{happy} \rrbracket) \\ &= (\lambda P. \lambda x. 0 \text{ iff } P(x) = 1)(\lambda y. 1 \text{ iff } y \text{ is happy}) \\ &= \lambda x. 0 \text{ iff } (\lambda y. 1 \text{ iff } y \text{ is happy})(x) = 1 \\ &= \lambda x. 0 \text{ iff } ((1 \text{ iff } x \text{ is happy}) = 1) \\ &= \lambda x. 0 \text{ iff } x \text{ is happy}\end{aligned}$$

- This shows semantics can sometimes apply similarly **above and below the word**.

Summary

- We have defined a general way to interpret many place predicates, using functions written in the λ -notation.
- This generalises the set-theoretic view, and could be extended to 3-place predicates (**ditransitives**), like *give*, *tell*, and causative *make*, although their syntax is a bit tricky.
- We will later see that the λ -notation allows us to create complex predicates from entire clauses, in the form of relative clauses.

Summary

- We have defined a general way to interpret many place predicates, using functions written in the λ -notation.
- This generalises the set-theoretic view, and could be extended to 3-place predicates (**ditransitives**), like *give*, *tell*, and causative *make*, although their syntax is a bit tricky.
- We will later see that the λ -notation allows us to create complex predicates from entire clauses, in the form of relative clauses.

Summary

- We have defined a general way to interpret many place predicates, using functions written in the λ -notation.
- This generalises the set-theoretic view, and could be extended to 3-place predicates (**ditransitives**), like *give*, *tell*, and causative *make*, although their syntax is a bit tricky.
- We will later see that the λ -notation allows us to create complex predicates from entire clauses, in the form of relative clauses.

Non-prototypical transitive constructions

Structural vs Thematic

- The concepts of Subject and Object in transitive constructions are mainly **structural**: Subject=Spec-TP and Object=Comp-V.

Structural vs Thematic

- The concepts of Subject and Object in transitive constructions are mainly **structural**: Subject=Spec-TP and Object=Comp-V.
- It's tempting to equate Subjects with the more semantic concept of **AGENTS** (initiating an action), and Objects with **THEMES** (undergoing an action).

(4) Sachou chases the moth.
AGENT THEME

Structural vs Thematic

- The concepts of Subject and Object in transitive constructions are mainly **structural**: Subject=Spec-TP and Object=Comp-V.
- It's tempting to equate Subjects with the more semantic concept of **AGENTS** (initiating an action), and Objects with **THEMES** (undergoing an action).

(4) Sachou chases the moth.
AGENT THEME

- But is it always the case? Can we find SVO sentences in English, where S is non-agentive?

Structural vs Thematic

- The concepts of Subject and Object in transitive constructions are mainly **structural**: Subject=Spec-TP and Object=Comp-V.
- It's tempting to equate Subjects with the more semantic concept of **AGENTS** (initiating an action), and Objects with **THEMES** (undergoing an action).

(4) Sachou chases the moth.
AGENT THEME

- But is it always the case? Can we find SVO sentences in English, where S is non-agentive?

(5) The mouse is/got chased (by Sachou). (Get-)Passive
THEME AGENT

(6) Sachou frightens the mouse. Theme-Experiencer psych predicate
THEME EXPERIENCER

(7) The moth fears Sachou. Experiencer-Theme psych predicate
EXPERIENCER Theme

Thematic (θ) roles

- We call thematic (“**theta**”) roles the more semantic roles assigned by a head to its **arguments** (complement/specifier in the head’s own projection).
- For instance, *eat* assigns THEME to its Object (complement) and AGENT to its Subject.⁶ This specification is called the θ -**grid** of *eat*.
- Passives reverse the prototypical structural hierarchy of θ -roles: THEME gets promoted to Subject, while AGENT is demoted to an adjunct position inside a *by*-phrase, which sometimes remains implicit.
- *Fear* assigns EXPERIENCER to its Subject and THEME to its Object. What about *frighten*?
- The θ -**criterion** states that every argument in a sentence gets exactly one θ -role, and every θ -role in the predicate’s θ -grid must be assigned to exactly one argument.

⁶This is a theory-internal argument for having the Subject start out in Spec-VP.

Thematic (θ) roles

- We call thematic (“**theta**”) roles the more semantic roles assigned by a head to its **arguments** (complement/specifier in the head’s own projection).
- For instance, *eat* assigns THEME to its Object (complement) and AGENT to its Subject.⁶ This specification is called the θ -**grid** of *eat*.
- **Passives reverse the prototypical structural hierarchy** of θ -roles: THEME gets promoted to Subject, while AGENT is demoted to an adjunct position inside a *by*-phrase, which sometimes remains implicit.
- *Fear* assigns EXPERIENCER to its Subject and THEME to its Object. What about *frighten*?
- The θ -**criterion** states that every argument in a sentence gets exactly one θ -role, and every θ -role in the predicate’s θ -grid must be assigned to exactly one argument.

⁶This is a theory-internal argument for having the Subject start out in Spec-VP.

Thematic (θ) roles

- We call thematic (“**theta**”) roles the more semantic roles assigned by a head to its **arguments** (complement/specifier in the head’s own projection).
- For instance, *eat* assigns THEME to its Object (complement) and AGENT to its Subject.⁶ This specification is called the θ -**grid** of *eat*.
- **Passives reverse the prototypical structural hierarchy** of θ -roles: THEME gets promoted to Subject, while AGENT is demoted to an adjunct position inside a *by*-phrase, which sometimes remains implicit.
- *Fear* assigns EXPERIENCER to its Subject and THEME to its Object. What about *frighten*?
- The θ -**criterion** states that every argument in a sentence gets exactly one θ -role, and every θ -role in the predicate’s θ -grid must be assigned to exactly one argument.

⁶This is a theory-internal argument for having the Subject start out in Spec-VP.

Thematic (θ) roles

- We call thematic (“**theta**”) roles the more semantic roles assigned by a head to its **arguments** (complement/specifier in the head’s own projection).
- For instance, *eat* assigns THEME to its Object (complement) and AGENT to its Subject.⁶ This specification is called the θ -**grid** of *eat*.
- **Passives reverse the prototypical structural hierarchy** of θ -roles: THEME gets promoted to Subject, while AGENT is demoted to an adjunct position inside a *by*-phrase, which sometimes remains implicit.
- *Fear* assigns EXPERIENCER to its Subject and THEME to its Object. What about *frighten*?
- The θ -**criterion** states that every argument in a sentence gets exactly one θ -role, and every θ -role in the predicate’s θ -grid must be assigned to exactly one argument.

⁶This is a theory-internal argument for having the Subject start out in Spec-VP.

Intransitive argument structures

Agentivity in intransitives

- Once again, it's tempting to believe intransitive predicates assign to their unique argument the role of AGENT. But **are all intransitives really agentive?**
- Let's try to modify the following sentence with *intentionally* to diagnose agency:⁷

(8) Jo intentionally smiled/jumped/protested.

(9) Jo (??intentionally) fell/arrived/died.

(10) The ice (*intentionally) froze/melted/broke.

- So not all intransitives are agentive! Some structural subjects seem to behave more like THEMES with certain verbs. These verbs are called **unaccusatives**(9-10).
- More “prototypical” intransitives that assign AGENT to their subject are called **unergative** (8).

⁷Another test is purpose clauses starting with *in order to*...

Agentivity in intransitives

- Once again, it's tempting to believe intransitive predicates assign to their unique argument the role of AGENT. But **are all intransitives really agentive?**
- Let's try to modify the following sentence with *intentionally* to diagnose agency:⁷

(8) Jo intentionally smiled/jumped/protested.

(9) Jo (??intentionally) fell/arrived/died.

(10) The ice (*intentionally) froze/melted/broke.

- So not all intransitives are agentive! Some structural subjects seem to behave more like THEMES with certain verbs. These verbs are called **unaccusatives**(9-10).
- More “prototypical” intransitives that assign AGENT to their subject are called **unergative** (8).

⁷Another test is purpose clauses starting with *in order to*...

Agentivity in intransitives

- Once again, it's tempting to believe intransitive predicates assign to their unique argument the role of AGENT. But **are all intransitives really agentive?**
- Let's try to modify the following sentence with *intentionally* to diagnose agency:⁷

(8) Jo intentionally smiled/jumped/protested.

(9) Jo (??intentionally) fell/arrived/died.

(10) The ice (*intentionally) froze/melted/broke.

- So not all intransitives are agentive! Some structural subjects seem to behave more like THEMES with certain verbs. These verbs are called **unaccusatives**(9-10).
- More “prototypical” intransitives that assign AGENT to their subject are called **unergative** (8).

⁷Another test is purpose clauses starting with *in order to*...

Agentivity in intransitives

- Once again, it's tempting to believe intransitive predicates assign to their unique argument the role of AGENT. But **are all intransitives really agentive?**
- Let's try to modify the following sentence with *intentionally* to diagnose agency:⁷

(8) Jo intentionally smiled/jumped/protested.

(9) Jo (??intentionally) fell/arrived/died.

(10) The ice (*intentionally) froze/melted/broke.

- So not all intransitives are agentive! Some structural subjects seem to behave more like THEMES with certain verbs. These verbs are called **unaccusatives**(9-10).
- More “prototypical” intransitives that assign AGENT to their subject are called **unergative** (8).

⁷Another test is purpose clauses starting with *in order to*...

Agentivity in intransitives

- Once again, it's tempting to believe intransitive predicates assign to their unique argument the role of AGENT. But **are all intransitives really agentive?**
- Let's try to modify the following sentence with *intentionally* to diagnose agency:⁷

(8) Jo intentionally smiled/jumped/protested.

(9) Jo (??intentionally) fell/arrived/died.

(10) The ice (*intentionally) froze/melted/broke.

- So not all intransitives are agentive! Some structural subjects seem to behave more like THEMES with certain verbs. These verbs are called **unaccusatives**(9-10).
- More “prototypical” intransitives that assign AGENT to their subject are called **unergative** (8).

⁷Another test is purpose clauses starting with *in order to*...

Agentivity in intransitives

- Once again, it's tempting to believe intransitive predicates assign to their unique argument the role of AGENT. But **are all intransitives really agentive?**
- Let's try to modify the following sentence with *intentionally* to diagnose agency:⁷

(8) Jo intentionally smiled/jumped/protested.

(9) Jo (??intentionally) fell/arrived/died.

(10) The ice (*intentionally) froze/melted/broke.

- So not all intransitives are agentive! Some structural subjects seem to behave more like THEMES with certain verbs. These verbs are called **unaccusatives**(9-10).
- More “prototypical” intransitives that assign AGENT to their subject are called **unergative** (8).

⁷Another test is purpose clauses starting with *in order to*...

Agentivity in intransitives

- Once again, it's tempting to believe intransitive predicates assign to their unique argument the role of AGENT. But **are all intransitives really agentive?**
- Let's try to modify the following sentence with *intentionally* to diagnose agency:⁷

(8) Jo intentionally smiled/jumped/protested.

(9) Jo (??intentionally) fell/arrived/died.

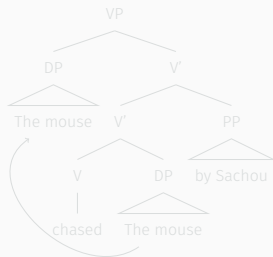
(10) The ice (*intentionally) froze/melted/broke.

- So not all intransitives are agentive! Some structural subjects seem to behave more like THEMES with certain verbs. These verbs are called **unaccusatives**(9-10).
- More “prototypical” intransitives that assign AGENT to their subject are called **unergative** (8).

⁷Another test is purpose clauses starting with *in order to*...

Keeping θ -assignment “uniform”

- Unaccusatives (and passives) show us that **the intuitive subject-AGENT object-PATIENT equivalence breaks down with certain verbs/structures, at least on the surface.**
- Is there are way to maintain the equivalence at a more basic level? Yes, movement!⁸

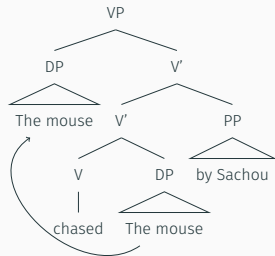


- Movement explains why THEMES can surface in subject positions.

⁸Minimalism sees movement as a re-MERGE operation: instead of merging an external element in Spec-VP, you merge an element already present in the tree (here, in Comp-V).

Keeping θ -assignment “uniform”

- Unaccusatives (and passives) show us that **the intuitive subject-AGENT object-PATIENT equivalence breaks down with certain verbs/structures, at least on the surface.**
- Is there are way to maintain the equivalence at a more basic level? Yes, movement!⁸

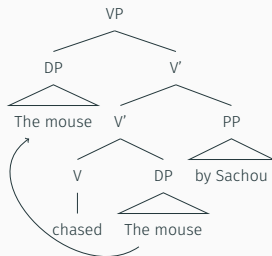


- Movement explains why THEMES can surface in subject positions.

⁸Minimalism sees movement as a re-MERGE operation: instead of merging an external element in Spec-VP, you merge an element already present in the tree (here, in Comp-V).

Keeping θ -assignment “uniform”

- Unaccusatives (and passives) show us that **the intuitive subject-AGENT object-PATIENT equivalence breaks down with certain verbs/structures, at least on the surface.**
- Is there are way to maintain the equivalence at a more basic level? Yes, movement!⁸



- Movement explains why THEMES can surface in subject positions.

⁸Minimalism sees movement as a re-MERGE operation: instead of merging an external element in Spec-VP, you merge an element already present in the tree (here, in Comp-V).

References i



Schönfinkel, M. (1924).Über die bausteine der mathematischen logik. *Mathematische Annalen*, 92, 305–316. <https://doi.org/10.1007/BF01448013>



Church, A. (1941). *The calculi of lambda-conversion*. Princeton University Press.



Curry, H. B., & Feys, R. (1958). *Combinatory logic* (Vol. 1). North-Holland.



Greenberg, J. H. (1963). *Universals of language*. MIT Press.



Gruber, J. S. (1965). *Studies in lexical relations* [Doctoral dissertation, Massachusetts Institute of Technology].



Montague, R. (1974). *Formal philosophy: Selected papers of richard montague* (R. H. Thomason, Ed.). Yale University Press.



Perlmutter, D. M. (1978). Impersonal passives and the unaccusative hypothesis. *Proceedings of the Fourth Annual Meeting of the Berkeley Linguistics Society*, 157–189.



Chomsky, N. (1981). *Lectures on government and binding*. Foris.



Stowell, T. (1981). *Origins of phrase structure* [Doctoral dissertation, Massachusetts Institute of Technology].



Burzio, L. (1986). *Italian syntax: A government-binding approach*. D. Reidel.



Dryer, M. S. (1992). The greenbergian word order correlations. *Language*, 68(1), 81–138.
<https://doi.org/10.1353/lan.1992.0028>



Haspelmath, M., Dryer, M. S., Gil, D., & Comrie, B. (Eds.). (2005). *The world atlas of language structures*. Oxford University Press.



Dryer, M. S., & Haspelmath, M. (Eds.). (2013). *The world atlas of language structures online*.



Biberauer, T., Holmberg, A., & Roberts, I. (2014). A syntactic universal and its consequences. *Linguistic Inquiry*, 45(2), 169–225.
https://doi.org/10.1162/LING_a_00153